

Module 7: Secure Software Supply Chain with SBOMs

Lab 1 Report

Student: Jaime | Repository: NG911-dev (TAMU)

Part 1: SBOM Generation

Two Software Bill of Materials (SBOMs) were generated for the **NG911-dev** repository (Texas A&M University), cloned into the GitHub Codespace environment. The tools used were **Syft** (by Anchore) and **Trivy** (by Aqua Security), each producing output in a different standard format.

Commands Executed

Tool	Command
Syft	syft . -o spdx-json > ../deliverables/sbom_syft_spdx.json
Trivy	trivy fs . --format cyclonedx --output ../deliverables/sbom_trivy_cdx.json

Component Count Results

Tool	Format	Components Detected
Syft	SPDX JSON	130
Trivy	CycloneDX JSON	104

Comparison: SPDX vs CycloneDX

The two formats differ in several key aspects. **SPDX** (Software Package Data Exchange), produced by Syft, is an ISO/IEC standard focused on license compliance and component provenance. It detected **130 components** and uses fields such as *SPDXID*, *licenseConcluded*, and *packageVerificationCode*. **CycloneDX**, produced by Trivy, is an OWASP standard optimized for security use cases and vulnerability management. It detected **104 components** and organizes data using *bom-ref*, *purl* (package URL), and dependency graphs. The difference in component count (26 more in Syft) reflects that Syft performs deeper filesystem inspection while Trivy focuses on known package manifests.

Part 2: Vulnerability Analysis

The Syft SBOM was fed into **Grype** (also by Anchore) to scan for known CVEs. Grype cross-references the component inventory against its vulnerability database and reports matches by severity.

Command Executed

```
grype sbom:../deliverables/sbom_syft_spdx.json -o table >
../deliverables/vuln_analysis_grype.txt
```

Grype reported: **5 vulnerability matches** — 0 critical, 1 high, 3 medium, 1 low, 0 negligible. All 5 are fixed in newer versions.

Top 5 Vulnerabilities

Vulnerability ID	Severity	Component	Installed	Fixed In
GHSA-vfmq-68hx-4jfw	High	lxml	5.2.2	6.1.0
GHSA-p423-j2cm-9vmq	Medium	cryptography	46.0.5	46.0.7
GHSA-6w46-j5rx-g56g	Medium	pytest	8.3.3	9.0.3
GHSA-m959-cc7f-wv43	Low	cryptography	46.0.5	46.0.6
GHSA-gc5v-m9x4-r6x2	Medium	requests	2.32.4	2.33.0

CVE Deep Dive: GHSA-vfmq-68hx-4jfw (lxml)

This is the only **High severity** finding. The vulnerability affects **lxml**, a Python library for processing XML and HTML. The flaw allows maliciously crafted XML input to cause unexpected behavior or information disclosure due to improper handling of certain XML structures. The NG911 application uses lxml version 5.2.2; upgrading to **6.1.0** fully mitigates this risk. Given that NG911 is public-safety software processing structured data, an XML parsing vulnerability could be exploited to disrupt emergency dispatch operations.

Part 3: Reflection

Security Engineering Principles Applied

This lab reinforced several core Security Engineering principles. **Open Design** was demonstrated through the use of open-source tools (Syft, Trivy, Grype) and standard formats (SPDX, CycloneDX), ensuring transparency and community auditability. **Complete Mediation** (Saltzer & Schroeder, 1975) was reflected in how Grype checks every component in the SBOM against its vulnerability database — not just a subset. **Defense in Depth** was illustrated by using two different tools (Syft + Trivy) and two different formats to cross-validate the component inventory, since each tool detected a different number of components (130 vs 104).

Insights and Observations

Generating SBOMs for the NG911 repository revealed that even a relatively focused Python-based application carries over 100 third-party dependencies. Of these, 5 had known vulnerabilities — all in Python libraries (lxml, cryptography, pytest, requests). Notably, all vulnerabilities have available fixes, meaning a simple dependency update would eliminate all identified risks. This highlights the value of automated SBOM generation in CI/CD pipelines: catching outdated dependencies before deployment rather than after a security incident.

Limitations

The SBOM analysis has limitations. Neither Syft nor Trivy can detect vulnerabilities introduced through custom code logic — only known CVEs in third-party packages. Additionally, the discrepancy in component counts between tools (130 vs 104) suggests that no single tool provides a complete picture, reinforcing the need for a defense-in-depth approach using multiple scanners.

References

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>

National Vulnerability Database. <https://nvd.nist.gov>

Anchore Syft. <https://github.com/anchore/syft>

Aqua Security Trivy. <https://github.com/aquasecurity/trivy>

Anchore Grype. <https://github.com/anchore/grype>